

Vergleich der Performanz

Benchmark-Setup

Um die in Abschnitt ?@sec-methoden vorgestellten Datenverarbeitungsmethoden zu bewerten, haben wir sie in einem systematischen Benchmark-Experiment auf dem SAS-Server des Statistischen Bundesamtes und den neuen OnSite R- und Stata-Servern des FDZ erprobt. Weitere Benchmarks sowie der vollständig reproduzierbare Code sind auf opencode.de verfügbar¹.

Betrachtet wurden simulierte Einzeldaten in verschiedenen Größen (0.1%, 1%, 10% der insgesamt 80 Mio Beobachtungen des BTP). Sie wurden aus öffentlich verfügbaren Informationen generiert, um die wesentlichen Strukturen des BTP für unsere Analyse künstlich nachzubilden. Abgrenzend zu den im FDZ bereitgestellten *Datenstrukturfiles* wurden die Testdaten nicht aus echten Mikrodaten abgeleitet. Der simulierte Datensatz ist ausschließlich für technische Testzwecke der nachfolgenden Anwendungsfälle bestimmt:

1. **Fallzahl** (pro Statistik und Jahr)
2. (logistische) **Regression** (Einflussfaktoren auf positive Verlustvorträge)

Jede Datenverarbeitungsmethoden wurde im Rahmen des Experiments dreifach angewandt mit dem Ziel der zuverlässigen Performanz-Messung. Verglichen werden die Methoden anhand der mittleren Messungen der folgenden Zielgrößen:

1. beanspruchter **Arbeitsspeicher** der Auswertung,
2. **Laufzeit** der Auswertung (real verstrichene Zeit),
3. **Festplattenspeicher** der verwendeten Daten.

Die Zielgrößen sind in dieser Reihenfolge zu priorisieren, da ein übermäßiger Arbeitsspeicherverbrauch zu schwerwiegenden Abbrüchen führen kann, während längere Laufzeiten dennoch zu einem erfolgreichen Ergebnis führen können. Festplattenspeicher ist ausreichend vorhanden, aber unnötige Belegung sollte dennoch vermieden werden.

Abstufungen der Zielgrößen sind für unsere Aufgaben nicht gleichermaßen relevant. Eine Methode, die die betrachteten Auswertungen in unter 10 Sekunden mit weniger als 4 GB Arbeitsspeicher durchführt, wird bereits als optimal bewertet; weitere Optimierungen sind in diesem Fall nicht erforderlich. Unter diesen Bedingungen, gilt es die Aufwände der Entwicklung zu reduzieren, sodass Aspekte, wie die einfache Anwendbarkeit und Nachvollziehbarkeit – selbst für unerfahrene Nutzende – entscheidend sind.

Wir betrachten in R (Version 4.5.0) die folgenden R-Pakete gemeinsam mit den in Abschnitt ?@sec-methoden beschriebenen Methoden (selektives Einlesen, Datenaufteilung):

¹www.opencode.de/

Table 1: Versionen betrachteter R-Pakete

Paket	Version
data.table	1.17.8
tidyverse	—
vroom	—
arrow	20.0.0.2
duckdb	1.3.2
polars	1.0.1

Table 2: Baseline anhand SAS und Stata (1% simuliertes BTP)

Software / Fall	RAM	Zeit	Disk
SAS Fallzahl	tba	2.43 Min	33 GB
SAS Regression	tba	tba	tba
Stata Fallzahl	ca. 33 GB	55 Sek	32 GB
Stata Regression	ca. 33 GB	56 Sek	32 GB

Wie in Abschnitt **?@sec-infrastruktur** beschrieben war es nicht möglich, die neuste Version aller Pakete zu verwenden und die leichter nutzbaren Variationen `{duckplyr}` bzw. `{tidytable}` zu nutzen. Die Package wurden aber wenige Tage nach dem Experiment eingerichtet. Die Syntaxtranslation kann dabei die Performanz verschlechtern, sodass bei `{duckdb}` bzw. `{data.table}` zwischen Performanz und Code-Verständlichkeit abgewägt werden muss. `{arrow}` ist schon länger als `{duckdb}` und `{polars}` verfügbar, sodass wir darin bereits mehr Erfahrungen aufbauen konnten.

Baseline

Klassisch werden die betrachteten Auswertungsaufgaben im Statistischen Bundesamt hauptsächlich mit SAS und im FDZ mit Stata gelöst. Um die folgenden Ergebnisse einzuordnen, haben wir die folgenden Performanzmessungen für 1% des simulierten BTP durchgeführt (800.000 Beobachtungen):

Der Arbeitsspeicher des FDZ OnSite-Servers reicht nicht aus, um das ganze BTP in Stata zu verarbeiten, sodass zwingend mit selektivem Einlesen gearbeitet werden muss.

Hauptergebnis für große Datenmengen

Anhand unserer Untersuchung, bietet `{arrow}+{parquet}` (bei Bedarf `+{duckdb}`) im Gesamtpaket die eindeutige besten Datenverarbeitungsmethoden für unsere Zwecke.

- Das geringe Arbeitsspeicherprofil Bedarf keiner weiteren Optimierung und ist in unserer Auswertung nahezu konstant abhängig von der Datenmenge, sodass die Methoden auf

Table 3: Benchmarkergebnisse für Berechnung der Fallzahl (10% simuliertes BTP)

Methode	Datenformat	Optimierung	RAM	Laufzeit
arrow	Parquet aufgeteilt (10 Dateien)	—	220 MB	4.7 Sek
duckdb	Ein Parquet	—	—	5.1 Sek
arrow	Ein Parquet	Selektiv	—	7.47 Sek
arrow	Parquet pro Jahr	—	—	11.73 Sek
arrow	Ein Parquet	—	—	20.61 Sek

noch größere Datenmengen skalieren können. Ansonsten bietet {duckdb} und {polars} stärkere Optimierung des Arbeitsspeichers.

- schnellste Laufzeit für große Datenmengen
- das Parquet Dateiformat reduziert den benötigten Festplattenspeicher massiv.
- beste Nutzererfahrung
 - Leicht und verständlich - adaptiert nativ schöne {tidyverse} Syntax
 - Flexibel, selektives Einlesen ist zwingend erforderlich
 - hohe Stabilität, fehlende Features können durch leicht in duckdb oder in den Arbeitsspeicher überführt und dennoch ausgeführt werden
- kleiner Nachteil: Erfahrung notwendig, da erst Datenaufteilung/selektives Einlesen die maximale Performanz erreichen

Unerwarteterweise ist {polars} für das simulierte 10% BTP abgeschlagen (unter den schlechtesten Methoden) und für andere Datenmengen optimal schnell.

Erkenntnisse für kleine Datenmengen

- Paket für die optimale Laufzeit nicht eindeutig
 - 1e6: arrow > duckdb > DT+SEL > polars
 - 1e5: polars > duckdb > arrow > DT+SEL
 - 1e4: polars > data.table > arrow > readr
- Dagegen ist das effizienteste betrachtete Datenformat über 1000 Beobachtungen eindeutig Parquet – sowohl in Speicherbedarf als auch in Laufzeit.
- Datenaufteilung bietet nur high-performance Methoden ab einer gewissen Datenmenge Vorteile. Für kleine Mengen überwiegt der Overhead durch das Lesen mehrerer kleiner Dateien. Klassische Methoden werden durch eine Datenaufteilung nur negativ beeinflusst.

- Die Ergänzung von selektivem Einlesen ist immer vorteilhaft. High-Performance Methoden können aber bei einer aufgeteilten Datenhaltung darauf verzichten.
- Von den klassischen Methoden ist DT+SEL am effizientesten, aber für unseren größten Anwendungsfall. Ein geschickter DTler kann mit moderaten Datenmengen auch sehr effizient arbeiten. Anfängern ist eher `{arrow}` zu empfehlen.
 - 5x langsamer als `{arrow}`
 - SEL notwendig, da ohne 3x größer als die Eingangsdaten, dauert 2.2 min anstatt 23 Sec. -> schwierige Anwendung (Entscheidung relevanter Variablen muss vorab getroffen werden unter Beachtung der Arbeitsspeicher Restriktionen)
 - SPLIT verschlechtert DT im Gegensatz zu arrow
 - `{data.table}`+csv ist die beste “klassische” R-Methode und erreicht maximal 23.58s, in der größten Ordnung wie die nicht optimierte Arrow Performance. Das die beste gemessene Performance für CSV (aufgeteilte Parquet sind doppelt so schnell). D.h. ein `{data.table}` Profi, ist besser als ein schlechter Arrow Programmierer, der aber größeres Potenzial für Verbesserungen hat.
- base+readr sind immer weit abgeschlagen

Ergebnisse

- – Für `{data.table}` ist die geschickte Programmierung noch viel wichtiger. Arrow wird höchstens langsam und beansprucht viel weniger RAM. Wohingegen data.table für den gesamten Datensatz (ohne Sel) das 600x an RAM belegt wie Arrow, damit viel schneller Abbricht und den schlechten Programmierer Abbrüche beschert.
 - * ohne Vorauswahl bricht DT ab, wenn das gesamte BTP verwendet werden soll. Arrow nur, wenn die Auswertung auch dazu führt, dass wirklich alle Variablen verarbeitet werden.
 - * mit DT lässt sich nicht so flexibel/schön arbeiten
- Arrow verzeiht Fehler leichter als DT. Alleine durch die Datenaufteilung (vorgegeben durch die Datenbereitstellung, vorbereitet durch das FDZ), erreichen wir Count Ergebnisse in 11.73s, nur 5s über dem absoluten Optimum.
- Wenn noch weniger RAM einsetzbar ist, benötigt Polars den wenigsten RAM, DuckDB nur geringfügig mehr

10 %: arrow + aufgeteilte parquet, duckdb, arrow variante 1 %: polars + ein parquet, duckdb, arrow 0.1 %: polars + ein parquet, DT, arrow 0.01%: DT + csv, readr, polars

-> nicht eindeutig. Für kleinere Datenmengen mehr Auswahl und das vertraute auszuwählen. Für größere Datenmengen klare Empfehlung Arrow, w.g. einfacher Anwendung, Stabilität (ggü.

Nutzerfehlern), Skalierbarkeit an die RAM-Grenze und darüber hinaus, höchste Performance für unsere Datenmengen+gute System Ressourcen.

-> für kleine Datenmengen macht chunking keinen Sinn.

Dateitypen:

Beide Anwendungen lassen sich in unter 20 Z. mit Arrow programmieren (verständlich lesbar) s. Opencode Repo.

RAM Wachstum Arrow, Polars, DuckDB, Data.table, readr, baser mit und ohne sel.

512 im FDZ Obergrenze.

Gewinne ggü. Klassischen Methoden 7.X Min DT und sehr viel RAM im Default. Dennoch nicht komplett davon abraten.

Fazit (nach Hinten)

- Empfehlung (insb. für unerfahrene Nutzende): Arrow, stabil, flexibel, einfach zu nutzen
- Profi's in `data.table`, die sehr vertraut sind mit der Syntax, müssen nur in Randfällen wechseln. Falls die wirklich genutzten Variablenauswahl nicht durch den verfügbaren Arbeitsspeicher bedient werden können (dt brauchte ca. 3x RAM ggü. Festplatte).
- neuste, beste, experimentell, bietet **polars** für moderate Datenmengen die effizienteste Variante
- wer mit sql vertraut ist oder Features außerhalb des Scopes von Arrow braucht (auf großen Inputs), nutzt auch gerne duckdb